

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return *the **wealth** that the richest customer has*.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Explanation:

1st customer has wealth = $1 + 2 + 3 = 6$

2nd customer has wealth = $3 + 2 + 1 = 6$

Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: `accounts = [[1,5],[7,3],[3,5]]`

Output: 10

Explanation:

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: `accounts = [[2,8,7],[7,1,3],[1,9,5]]`

Output: 17

Constraints:

- $m == \text{accounts.length}$
- $n == \text{accounts}[i].\text{length}$
- $1 \leq m, n \leq 50$

- $1 \leq \text{accounts}[i][j] \leq 100$

Program :

```
class Solution {  
public:  
    int maximumWealth(vector<vector<int>>& accounts) {  
        int maxWealth = 0;  
  
        for (int i = 0; i < accounts.size(); i++) {  
            int wealth = 0;  
  
            for (int j = 0; j < accounts[i].size(); j++) {  
                wealth += accounts[i][j];  
            }  
  
            maxWealth = max(maxWealth, wealth);  
        }  
  
        return maxWealth;  
    }  
};
```

Output :

☒ Testcase | [>_ Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

Output

6

Expected

6